OncoGemma v4 — Staged PRD: Overview

Date: 22 Aug 2026 · **Builder:** 1 engineer · **Strategy:** 7 incremental stages, each ends with a working end-to-end system. No stage depends on unfinished future work.

Umbrella document: ../OncoGemma-v4-PRD.md. This directory is the build plan.

---

1. Stage map

Stage	Name	E2E outcome when done	Est. (solo)
v4.0	Walking skeleton	Upload a WSI → view it in the browser at full zoom	1 wk
v4.1	Preprocess + QC gate	Stain-normalized layer + bad slides rejected with reasons	1 wk
v4.2	Hotspot triage	Tumor heatmap + editable hotspot ROIs, confirm gate	1.5 wk
v4.3	Mitosis counting	Detections + 10 virtual HPFs + live mitotic score	2 wk
v4.4	Nottingham grading	Tubule + pleomorphism sub-scores → deterministic grade	1.5 wk
v4.5	CAP report + sign-off	Signed, immutable, server-rendered PDF dossier	1 wk
v4.6	Validation + hardening	Batch harness vs archive, override analytics, cost logs	1–2 wk

Total: ~9–10 weeks solo. Stages are strictly sequential; each is demo-able to a pathologist.

2. Locked decisions (from spec review, 22 Aug)

1. **Mitotic score = mitoses/mm²** mapped through a configurable CAP table (default thresholds derived from Elston-Ellis 2.74 mm²: score 2 ≥ 3.65/mm², score 3 ≥ 7.30/mm²). Classic per-10-HPF count also displayed.
2. **Mitosis verifier = dedicated classifier**, published weights first; HoVer-Net deferred (morphometry is optional evidence, v4.6+).
3. **Histologic type = auto-proposed + mandatory confirm** (field blocked until pathologist acts).
4. **Fresh repo**; port v3 pieces (viewer config, prompts) selectively.
5. **Scanner support = generic**: anything OpenSlide reads (SVS, NDPI, MRXS, generic tiled TIFF) + DICOM WSI. No vendor-specific paths.
6. **Published model weights wherever they exist**; own training deferred (only exception: the tumor-probability linear probe — hours of work, unavoidable).
7. **Stain normalization = Macenko via tiatoolbox** in-pipeline. STAINS/Aequip and QuPath are out of the runtime path; QuPath remains the offline annotation tool for building validation ground truth.
8. **DICOM archival conversion deferred to v4.6** (interoperability, not user-facing; spec's Stage 2 requirement, consciously postponed).
9. **No cost ceiling**, but per-case cost logged from v4.2 (embedding calls dominate).
10. Signed resumable uploads, label/macro stripping at ingest, append-only audit log — from v4.0, non-negotiable.

3. Stack (pinned)

Layer	Choice	Why
Backend	Python 3.12, FastAPI, SQLAlchemy 2 + Alembic	ML ecosystem, one language for API + workers
DB	Postgres 16 (Cloud SQL; local: Docker)	state machine + geometry + JSONB
Queue	**None.** Workers poll Postgres with `FOR UPDATE SKIP LOCKED`	1 person, 6 stages; a queue table is debuggable, Pub/Sub is not.
Object store	GCS, uniform bucket-level access, `asia-south1`	PHI residency

Layer	Choice	Why
WSI I/O	OpenSlide 4.x + openslide-python; tiatoolbox 1.6+	generic format support; masking/normalization/patch utils
Pyramid	pyvips (`dzsave`) → DZI tiles	10–50× faster than Python tiling
Frontend	Next.js 15, OpenSeadragon 5, Annotorious (OSD plugin), Tailwind	viewer + polygon editing solved off the shelf
Auth	Firebase Auth (Google sign-in) + role column in Postgres	solo-friendly; SSO later
ML serving (Google)	Vertex AI endpoints: Path Foundation, MedGemma 1.5	usage-based, already provisioned
ML serving (ours)	Same worker container, GPU optional (Cloud Run GPU L4 for v4.3+)	published weights, inference-only
PDF	WeasyPrint (server-side)	pure Python, no headless browser to babysit
Deploy	Cloud Run (api, web, worker), Cloud SQL, GCS	zero-ops

4. Repo layout

```
oncogemma/
  backend/
    app/              # FastAPI: routers/, models/ (SQLAlchemy), schemas/ (pydantic)
    worker/           # poll loop + stage handlers: ingest.py, preprocess.py, qc.py,
                      #   triage.py, mitosis.py, grading.py, report.py
    pipeline/         # pure logic, no I/O: scoring.py, hpf.py, stain.py, qc_checks.py
```

```
  alembic/
  tests/
frontend/
  app/              # Next.js app router
  components/viewer/ # OSD wrapper, overlay layers, annotation tools
configs/
  scoring.yaml      # CAP tables, thresholds — all clinical constants live here
  cap_elements.yaml  # report element definitions (v4.5)
  prompts/          # versioned MedGemma templates (v4.4)
ops/                # Dockerfiles, cloudrun.yaml, terraform-lite (or gcloud scripts)
docker-compose.yml   # local: postgres + fake-gcs-server
```

Rule: `pipeline/` is pure functions (geometry in, scores out) — unit-testable without GCS/DB. Workers orchestrate; pipeline computes.

5. Shared contracts (all stages obey these)

5.1 Coordinate convention

All persisted geometry is in **base-level micrometers**, origin top-left of level 0. Slide row stores `mpp_x, mpp_y`. Conversions (`um ↔ level-N px`) live in `pipeline/coords.py` and nowhere else. Overlays convert at render time.

5.2 Stage state machine

queued → running → { awaiting_review | done | failed }

awaiting_review → confirmed → (next stage queued)

awaiting_review → rejected → (stage re-queued with edits applied)

One row per attempt in `stage_executions`. Machine output is written to GCS as one atomic JSON blob (`output_ref`); the DB row flips status last. Pathologist edits are stored as an

RFC-6902-style JSON diff in `review_edits` — machine output is never mutated. Confirmation snapshot = machine output + applied diff.

5.3 Core DDL (created in v4.0, extended per stage)

```
CREATE TABLE cases (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  created_by text NOT NULL, status text NOT NULL DEFAULT 'open',
  created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE slides (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  case_id uuid NOT NULL REFERENCES cases(id),
  gcs_uri_original text NOT NULL, gcs_uri_pyramid text,
  gcs_uri_pyramid_norm text, format text, scanner text,
  mpp_x double precision, mpp_y double precision,
  base_mag double precision, width_px int, height_px int,
  checksum_sha256 text, label_stripped_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE stage_executions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  case_id uuid NOT NULL REFERENCES cases(id),
  stage text NOT NULL,          -- ingest|preprocess|qc|triage|mitosis|grading|report
  attempt int NOT NULL DEFAULT 1,
  status text NOT NULL DEFAULT 'queued',
```

```
  input_ref jsonb, output_ref text, error text,
  model_versions jsonb, config_hash text,
  started_at timestamptz, completed_at timestamptz,
  reviewed_by text, reviewed_at timestamptz, review_edits jsonb,
  UNIQUE (case_id, stage, attempt));
CREATE INDEX ix_stage_poll ON stage_executions (status, stage) WHERE status = 'queued';
CREATE TABLE audit_events (
  id bigserial PRIMARY KEY,
  case_id uuid, actor text NOT NULL, event_type text NOT NULL,
  stage text, payload jsonb, created_at timestamptz NOT NULL DEFAULT now());
-- App role gets INSERT + SELECT only:
REVOKE UPDATE, DELETE ON audit_events FROM oncogemma_app;
```

5.4 Worker poll loop (v4.0, reused by every stage)

```
# worker/main.py — single process, one loop, dispatch by stage name
HANDLERS = {"ingest": ingest.run, "preprocess": preprocess.run, ...}
while True:
    with session.begin():
        row = session.execute(text("""
          SELECT id FROM stage_executions
          WHERE status='queued' AND stage = ANY(:stages)
          ORDER BY started_at NULLS FIRST LIMIT 1
          FOR UPDATE SKIP LOCKED"""), {"stages": list(HANDLERS)}).first()
```

```
        if row: mark_running(row.id)
    if not row: time.sleep(2); continue
    try:
        out_uri, model_versions = HANDLERS[stage](exec_row)   # writes JSON blob to GCS
        finish(row.id, out_uri, model_versions)               # → awaiting_review or done
    except Exception as e:
        fail(row.id, traceback.format_exc())
```

Idempotency rule: handlers derive all inputs from `input_ref` + DB, write output keyed by execution id — re-running a failed execution is always safe.

5.5 Config discipline

Every clinical constant (thresholds, field radii, CAP tables, prompt versions) lives in `configs/*.yaml`, loaded once, hashed (sha256 of canonical JSON) into `stage_executions.config_hash`. No clinical numbers in code.

5.6 Audit events (minimum set)

`case_created, slide_uploaded, stage_started, stage_output, stage_confirmed, stage_rejected, review_edit, score_override, report_signed, model_invocation` — `model_invocation` payload: `{model_id, version_or_weights_sha, endpoint, request_count, latency_ms, cost_estimate_usd}`.

6. Environments

- **local**: docker-compose (postgres:16, fake-gcs-server), `OPENSLIDE_PATH` via brew/apt, Vertex calls hit real endpoints with dev project. One `make dev` target runs api + worker + web.
- **dev / prod**: two GCP projects. Buckets: `og-{env}-raw`, `og-{env}-pyramids`, `og-{env}-artifacts`. All `asia-south1`, UBLA, no public access, 30-day lifecycle rule on `raw` scratch prefix only.

7. Definition of done (every stage)

1. E2E demo criterion met on ≥ 2 slides from different vendors (use public slides: TCGA-BRCA SVS + a Hamamatsu NDPI sample).
2. Unit tests green for everything in `pipeline/` touched by the stage.
3. New audit events emitted and visible via `GET /cases/{id}/audit`.
4. Deployed to dev environment (Cloud Run), not just laptop.
5. `configs/` diff reviewed — no clinical constant snuck into code.